Assignment: Responsive CSS Design with Flexbox

Topic: Responsive Design Principles and CSS Flexbox
Total marks: 100
Estimated time: 3 to 4 hours
Submission format: One folder containing `index.html`, `style.css`, and `answers.md` (Part A).

Instructions

- Write all CSS by hand. Do not use Bootstrap, Tailwind, or any CSS framework.
- Start your stylesheet with the `box-sizing: border-box` reset.
- Work **mobile-first**: write your base styles for small screens first, then enhance for larger screens.
- Prefer flexible units (`%`, `rem`, `vw`) over fixed `px` values wherever a value should adapt.
- Test your work by resizing your browser window and by using your browser's device toolbar (DevTools).
- Comment your CSS where you make an important decision.

Part A: Concept Questions (25 marks)

Answer in your own words in `answers.md`. Short, clear answers are better than copied definitions.

1. What does "responsive design" mean? Give two reasons why it matters. *(4 marks)*
2. Explain the "mobile-first" approach. Why do many developers prefer it to designing for desktop first? *(4 marks)*
3. Why do we prefer units like `%`, `rem`, and `vw` over `px` for values that should adapt? Give one situation where `px` is still a sensible choice. *(4 marks)*
4. What does `box-sizing: border-box` do, and why do we put it at the start of a stylesheet? *(4 marks)*
5. Flexbox lays out items in a single direction. What is the difference between the **main axis** and the **cross axis**, and how does `flex-direction: column` change them? *(5 marks)*
6. What is the difference between `align-items` and `align-content`? Under what condition does `align-content` have any effect? *(4 marks)*

Part B: Property Practice (20 marks)

Create a section in `index.html` with a `.container` holding **6 boxes** (each about `100px` wide and `80px` tall, different background colours, numbered 1 to 6). Then demonstrate each of the following. For each one, create a separate class (for example `.demo-1`, `.demo-2`) so all examples can be viewed on the same page, each labelled with a heading.

#	Demonstrate	Marks
1	Boxes in a row, spaced evenly with `justify-content: space-between`	3
2	Boxes stacked vertically using `flex-direction: column`	3
3	Boxes centred both horizontally and vertically inside a container of `300px` height	4
4	Boxes wrapping onto multiple lines using `flex-wrap: wrap`, with a `1rem gap`	4
5	Wrapped boxes using `align-content: space-between` inside a tall container (show that it works)	3
6	Boxes in `row-reverse` order, aligned to the bottom of the container using `align-items`	3

Under each demo, write one comment in the CSS explaining what happened and why.

Part C: Build a Responsive Page (45 marks)

Build a one-page website for a **fictional business of your choice** (for example a bakery, a barber shop, a tech school, or a photography studio). Use Flexbox for layout.

Required sections

1. **Header with navigation:** logo or business name on one side, links on the other.
2. **Hero section:** a heading, a short line of text, and a call-to-action button, centred.
3. **Services/Features section:** at least **4 cards**, each with a heading and short text.
4. **Testimonials or Team section:** at least 3 items side by side on large screens.
5. **Footer:** at least two columns of content (for example contact info and social links).

Layout requirements

Screen size	Expected behaviour
Phone (under 600px)	Everything stacked in a single column. Navigation links stack or wrap neatly.
Tablet (600px to 900px)	Cards in 2 per row.
Desktop (over 900px)	Navigation in one row. Cards 4 per row (or 3, if you have 3 or 6). Footer columns side by side.

Technical requirements

- Use `display: flex`, `flex-direction`, `flex-wrap`, `justify-content`, `align-items`, and `gap` meaningfully, not just for decoration.
- Use at least **two media queries** to adjust the layout at breakpoints. Keep them minimal, and let `flex-wrap` and `gap` do as much of the reflowing as possible.
- Use `rem` for font sizes and spacing, and `%` or `vw` for widths where appropriate.
- Include `<meta name="viewport" content="width=device-width, initial-scale=1.0">` in your HTML.
- The page must have **no horizontal scrollbar** at any width from 320px to 1400px.
- Use semantic HTML (`header`, `nav`, `main`, `section`, `footer`).

Part C: Debug It (10 marks)

The snippet below has **four problems**. Find and fix them in `answers.md`, and explain each fix in one sentence.

```
<div class="row">
  <div class="card">One</div>
  <div class="card">Two</div>
  <div class="card">Three</div>
  <div class="card">Four</div>
</div>
```

```css
.row {
  display: flex;
  flex-direction: row;
  align-content: center;
  justify-content: space-between;
  margin: 10px;
}

.card {
  width: 350px;
  height: 120px;
  padding: 20px;
  background: lightblue;
}
```

Goal: On a phone, the cards should wrap onto new lines with `1rem` spacing between them, the cards should not overflow the screen, and each card should be centred vertically within its row.

(Hint: think about wrapping, the unit used for width, `box-sizing`, and which property is being used for alignment.)

Submission Checklist

- [ ] `index.html`, `style.css`, and `answers.md` are included
- [ ] The page works at 320px, 768px, and 1200px widths
- [ ] There is no horizontal scrolling at any width
- [ ] Code is neatly indented and commented
- [ ] Parts A, B, and C are all complete

Bonus (up to 5 marks): Add a hamburger-style navigation that collapses on mobile using only CSS (the checkbox hack) or a small amount of JavaScript.